Group 4 - Milestone 1

Ishan Shivansh Bangroo Zhi-hong Jian Tara Liu Leona Pierce

Project repository `https://github.com/leona5139/choice_reversals`

1 Pilot design

We adapt the four treatments in Table 2 of Andreoni et al. (2020), using Section III and Appendix C. Each player completes eight rounds and assigns ten tickets between two households in a $10 donation scenario. The computer's allocations follow Table 3 of the paper. The differences from the human procedure are described below.

In this pilot experiment on LLMs, we ran five players per model in each treatment at temperature 1.0. The full player data cover 80 players. We used Recursive Flint (for free) and GPT-6 Luna (for about $0.02) through the OpenRouter API. We ran Gemini 3.1 Flash-Lite and Gemini 3.5 Flash-Lite (for free) through the Gemini API. The recorded model IDs are `recursive/flint-v1.0:free`, `openai/gpt-6-luna`, `gemini-3.1-flash-lite`, and `gemini-3.5-flash-lite`.

2 Prompts and task sequence

We use one fixed set of high-level prompts for all four treatments (Appendix B). The sequence of messages differs across treatments, as described below. Each player starts a fresh message thread. Before each API call, we add the information revealed so far and the current question. We send the full thread and append the reply with `"role": "assistant"` (see the player data examples).

We requested a single JSON object with integer ticket counts summing to ten. Of 713 saved replies, 37 did not follow that format. Gemini 3.5 Flash-Lite enclosed 26 replies in Markdown code fences, and Recursive Flint returned the same JSON object twice in 11 replies. The parser extracts the last object. All 713 saved replies yield valid allocations that match the stored decisions, including those 37 replies. API retry and failure totals are outside this check. Appendix C shows four actual prompts and replies with brief interpretations.

Each of the four main treatments has a different sequence of the three tasks below.

- **A.** The player allocates ten tickets. We then draw the winning ticket and reveal which group holds it.
- **P.** We draw the ticket and reveal its winning group first. The player allocates if their group wins. We then reveal the ticket number.
- A^R**.** The player submits all four allocations before any group is revealed. We then reveal the group one round at a time. When the player's group wins, they can keep or change that round's allocation.

All four treatments end with four A^R rounds. Rounds 1–4 are all A (`4A_4AR`), all P (`4P_4AR`), two A followed by two P (`2A2P_4AR`), or two P followed by two A (`2P2A_4AR`).

3 Adaptations and limitations

The pilot uses text and ticket counts in place of photographs and individual ticket buttons. We omitted recipient photographs, household-type descriptions, and timed breaks. The prompts retain the donation scenario, but no actual donations were made. We also omitted the probability display and review and confirmation loop. The human confirmation screen shows the households' winning probabilities and allows another revision before confirmation (Appendix Figure C.21). The pilot accepts one allocation without that step.

The code fixes exactly two participant wins in each block of four rounds. Before any outcomes are revealed, each round has a one-half chance of a participant win, but outcomes within a block are dependent. This gives every player two revision opportunities in rounds 5–8. The paper's aggregate results do not establish its exact draw schedule, so the fixed schedule remains a replication assumption to verify.

After a participant win, the prompts do not explicitly state the one-in-ten chance for each participant ticket. P outcomes show a ticket number without identifying its household, and P rounds won by the computer do not show its allocation. The pilot also reveals the winning group after ordinary A rounds. Whether humans received this feedback before the next round remains unverified. Before the main experiment, the next prompt version should restore the probability display and confirmation step, state the within-group probability, make P outcomes interpretable, and verify the draw schedule and A-round feedback.

4 Findings

Figure 1 compares human and LLM choices in rounds 1–4 of the pure A and P treatments, `4A_4AR` and `4P_4AR`. In the ex ante frame, players allocate before the draw. In the ex post frame, they allocate after learning their group won. An ex ante equalizing allocation offsets the computer's allocation so that each household holds ten of the twenty tickets.

- Even in the ex ante frame, models chose the 5/5 split in 39 of 80 choices (49%), compared with about 8% for humans.
- Choices varied by model (Figure 3). GPT-6 Luna chose the ex ante equalizing allocation in all 20 ex ante choices, matching the most common human category in that frame. It also chose that allocation in 9 of 10 ex post choices, whereas the most common human ex post choice was 5/5. Its agreement with humans therefore depends on the frame. Gemini 3.5 Flash-Lite chose 5/5 in 18 of 20 ex ante choices and 9 of 10 ex post choices.

In rounds 5–8 of the treatment that starts with four ex ante rounds, only GPT-6 Luna revised. It switched from the ex ante equalizing allocation to 5/5 in 9 of its 10 revision opportunities (Figure 2 and Figure 4).

Across all four models in that treatment, 9 of 40 eligible choices changed (22.5%), compared with the paper's 68.3% revision rate. Our displayed final panel keeps all 80 round records, including 40 computer-winning rounds without a revision opportunity. We therefore compare reversal rates using eligible opportunities and report eligible final choices separately. Of the 40 eligible final choices, 20 are 5/5 and 19 are ex ante equalizing; 11 of those 20 were already 5/5 before the reveal. Across all four treatments, 33 of 160 eligible choices changed. GPT-6 Luna made 30 of them, all to 5/5. The other three, two by Gemini 3.1 Flash-Lite and one by Recursive Flint, moved all or most tickets to one household.

These are descriptive pilot results from five players per model and treatment. Several decisions come from the same player, so they are not independent observations. The small sample and procedure differences limit conclusions about agreement with humans.

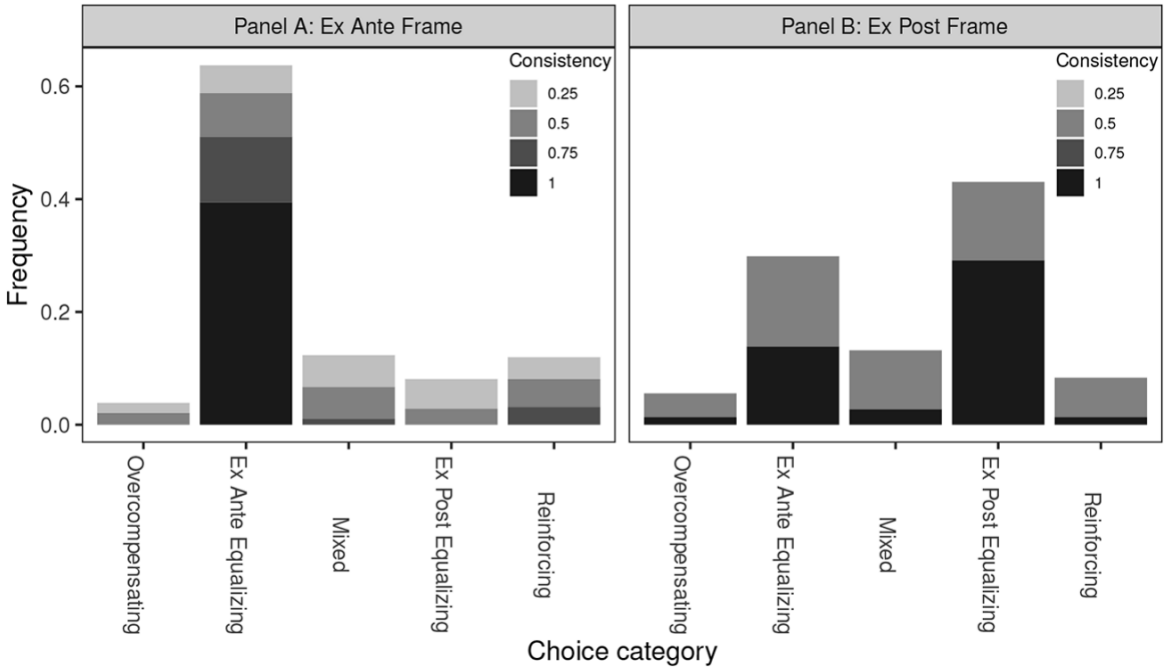
5 Division of work

Ishan Shivansh Bangroo prepared the initial paper-based prompt draft and an early report outline, checked the procedure against the paper, reviewed the saved replies and report, and added the reply-checking script. Tara Liu reviewed and revised the prompts and integrated the agreed wording. Leona Pierce developed the API flow, outcome messages, and Gemini runs. Zhi-hong Jian developed prompt-version and result tracking, ran the OpenRouter models, and prepared the analysis and report. Prompt design and review were shared tasks.

References

Andreoni, J., Aydin, D., Barton, B., Bernheim, B. D., and Naecker, J. (2020). When fair isn't fair. Understanding choice reversals involving social preferences. *Journal of Political Economy*, 128(5), 1673–1711. https://doi.org/10.1086/705549.

Human experiment (Andreoni et al., 2020, fig. 1)



LLM pilot

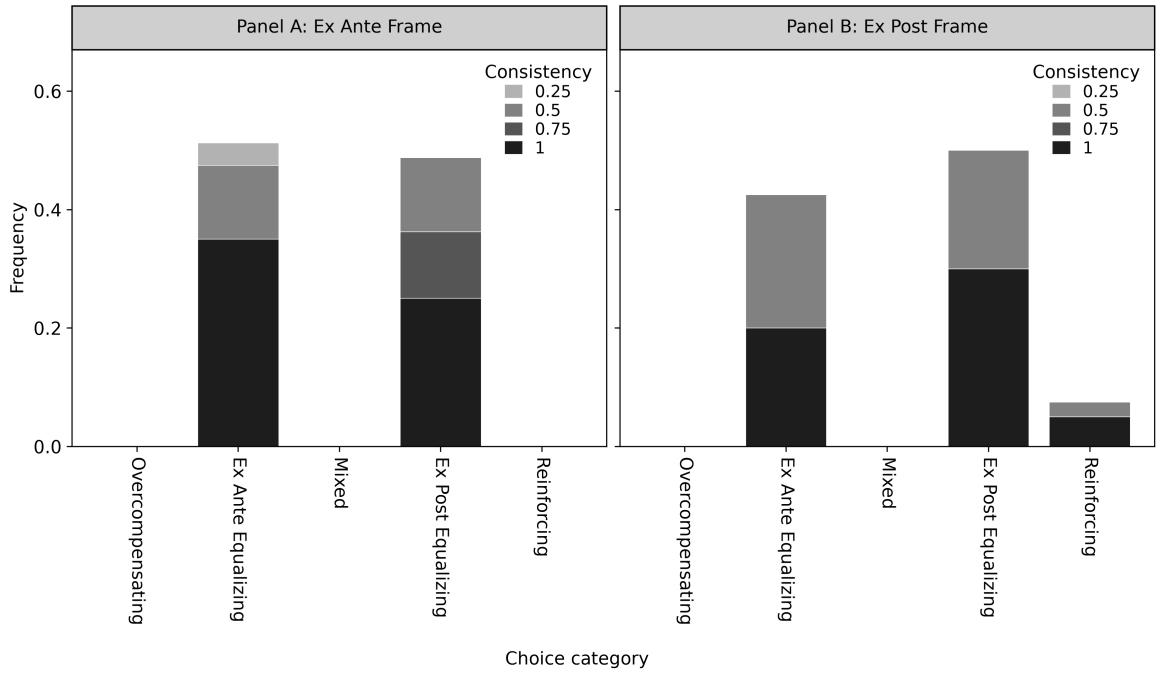
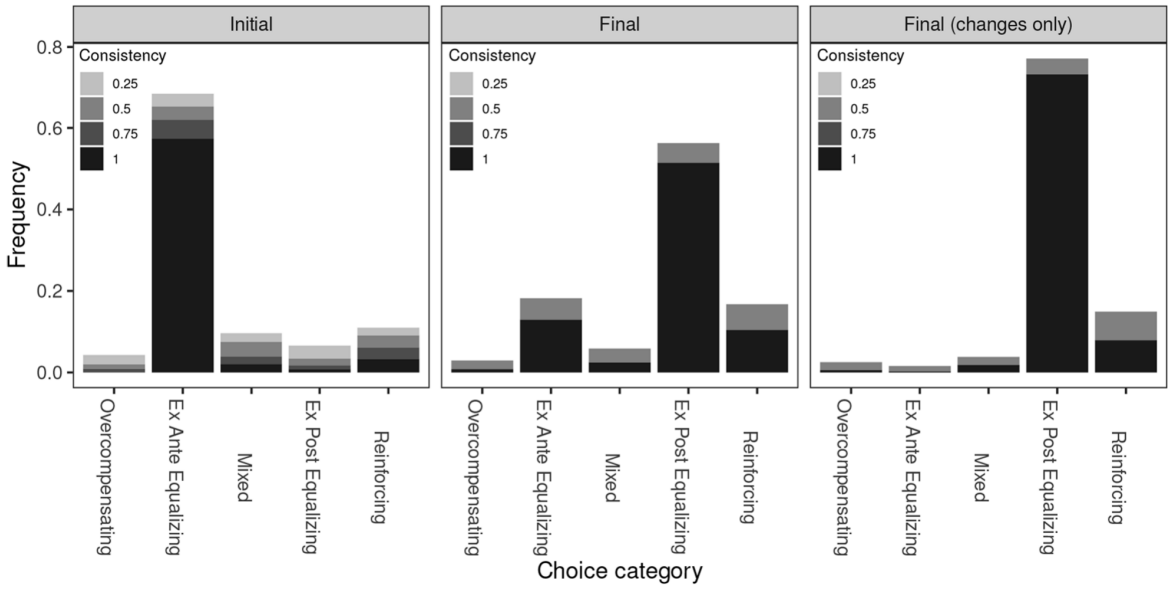


Figure 1: Allocations in A and P tasks, rounds 1–4.

Human experiment (Andreoni et al., 2020, fig. 3)



LLM pilot

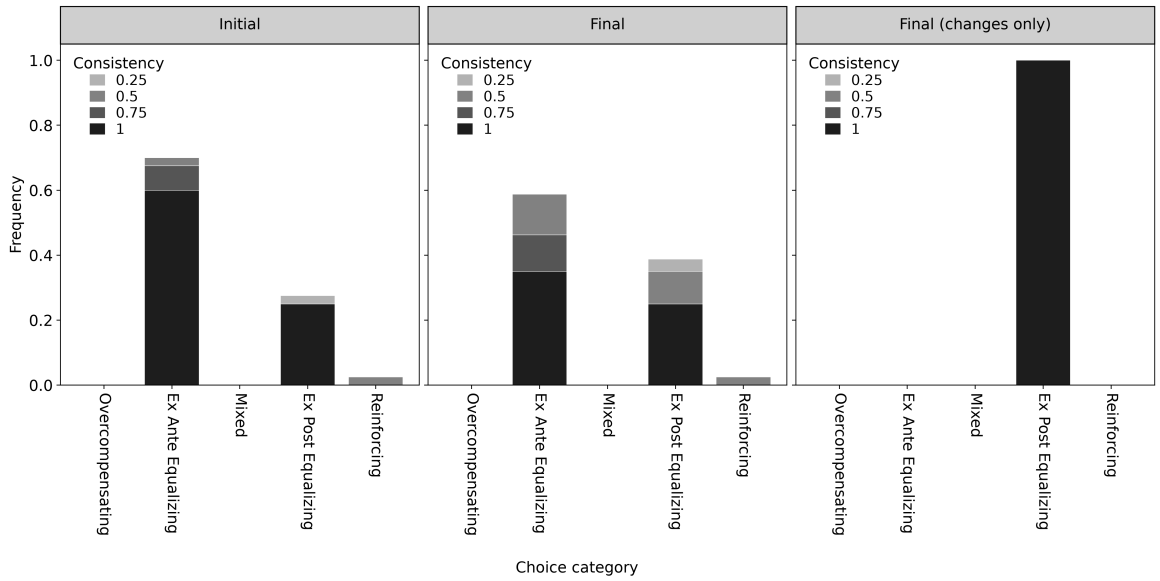


Figure 2: Allocations in 4A_4AR, rounds 5–8. Pilot initial and final panels each include 80 round records. The final panel retains the initial allocation in 40 computer-winning rounds with no revision opportunity. The changes-only panel contains 9 responses. The text separately compares eligible reversal rates and reports final choices for the 40 eligible rounds.

A Pilot results by model

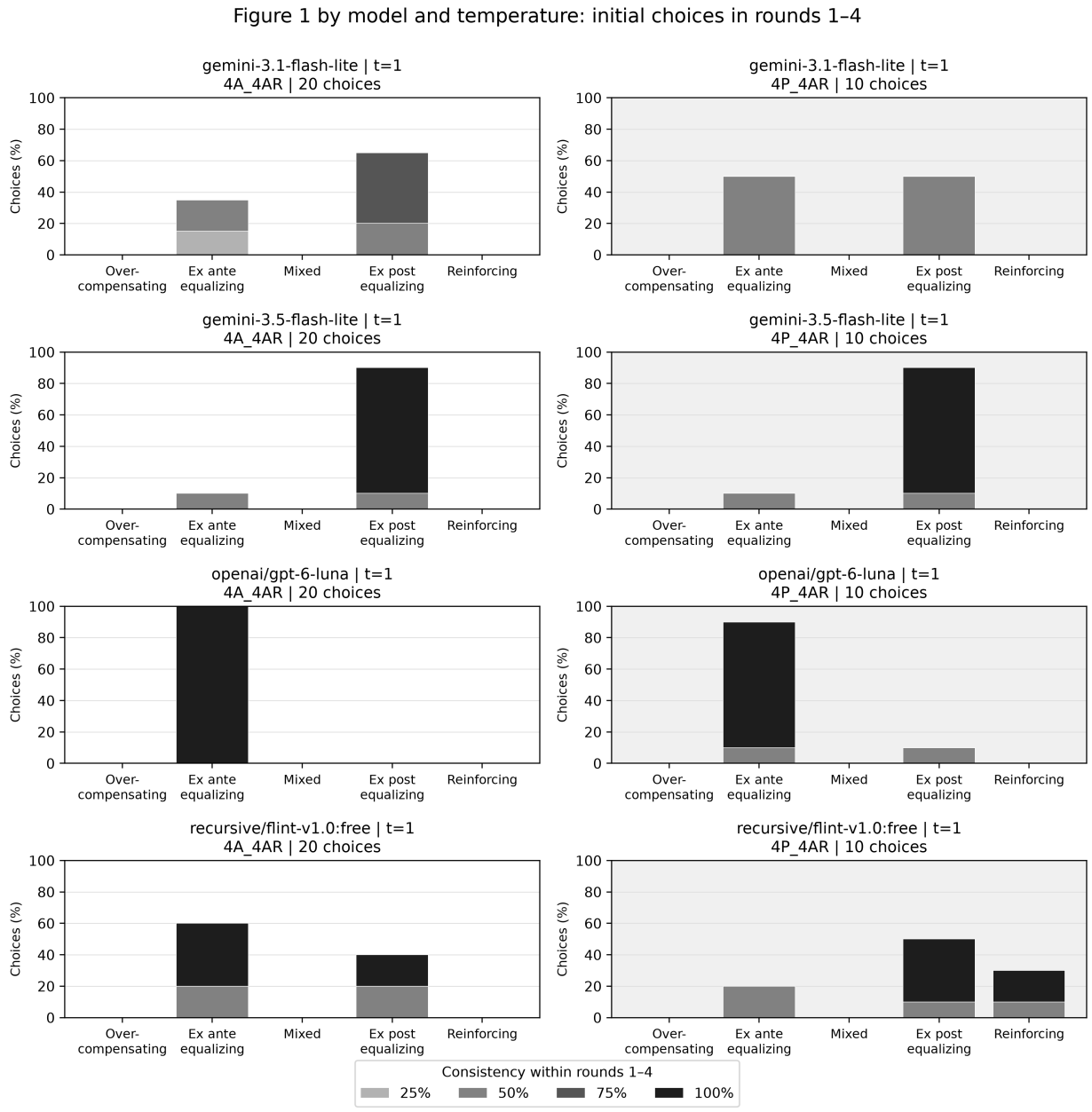


Figure 3: Initial allocations in A and P tasks by model.

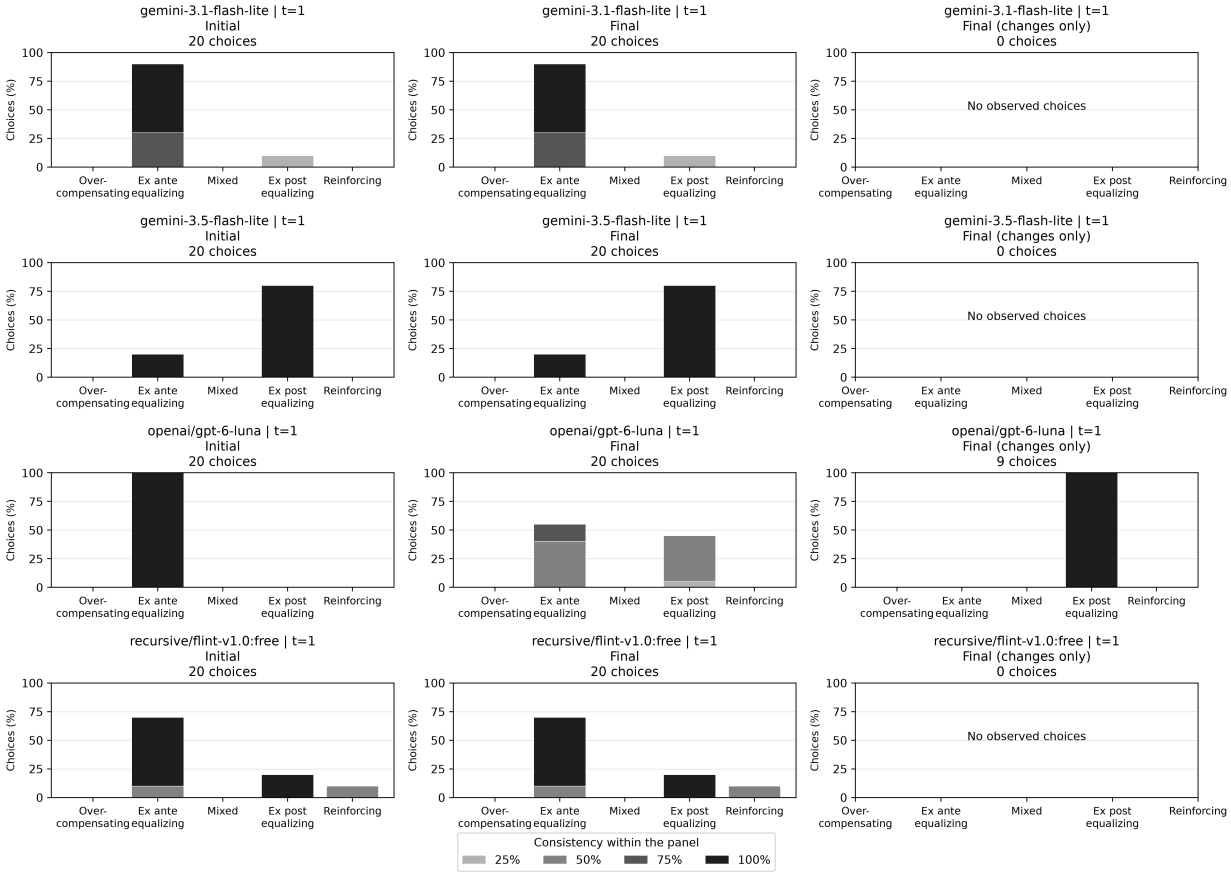


Figure 4: Initial and final allocations in 4A_4AR by model, rounds 5–8.

B Pilot prompt

```
[SYSTEM]
You are participating in study of allocation behaviors. Follow the instructions for each
  round and make your own decisions.

[INSTRUCTIONS]
In this study, you will decide how to allocate chances to win $10 donations among actual
  households in Kenya.

There are eight rounds. Each round concerns two different recipient households, called
  Household A and Household B. You will not be given individual information that
  distinguishes A from B or indicates that one is more deserving of a donation.

Your decisions concern tickets 1-10, which can be assigned between A and B. The computer
  has already assigned tickets 11-20. You will see the computer's allocation for that round
  before making any allocation decision, and you cannot change it. One of the 20 tickets is
  selected at random for each round as the winning ticket: the household holding that
  ticket is the winner for that round.

Only one of the eight rounds will actually count. After all eight rounds are complete, one
  round will be selected at random, and a $10 donation will be made to that round's winning
  household. Treat each round as one that could count.

[ALLOCATION]
Round {round}. In this round, the computer has already assigned {computer_a} of its 10
  tickets to Household A and {computer_b} to Household B. Its allocation cannot be changed.

Assign your 10 tickets between Household A and Household B based on the information you
  have received for this round.

[GROUP_REVEAL]
For this round (Round {round}), the winning ticket has been put in the {group} set of 10
  tickets (your tickets are 1-10; computer tickets are 11-20). Its specific number has not
  been revealed.

If it is in the computer set, the computer's existing allocation determines the winner and
  you have no further allocation decision for this round. If it is in your set, your
  allocation of tickets 1-10 can still determine which household wins.

[REVISION]
Earlier in Round {round}, you assigned {choice_a} of your tickets to Household A and
  {choice_b} to Household B. The computer assigned {computer_a} tickets to A and
  {computer_b} to B. You now know that the winning ticket is one of your tickets 1-10, but
  you do not know its specific number.

Review your earlier allocation. You may keep it or change it. Submit the allocation of your
  10 tickets that you want to use for this round. To keep your earlier choice, submit the
  same numbers again.

[OUTCOME_A]
Round {round} is complete. The winning ticket is in the {group} set of tickets.

[OUTCOME_P]
Round {round} is complete. The winning ticket is ticket {ticket}, in the {group} set of
  tickets.

[RESPONSE_FORMAT]
Respond with a single JSON object and nothing else: {"A": <number of your 10 tickets to
  Household A>, "B": <number of your 10 tickets to Household B>}. Use whole numbers from 0
  to 10 that sum to 10.
```

C Selected pilot prompts and responses

These four examples reproduce the current question and raw reply exactly from `output/results/pilot_1/players.jsonl`. Each call also included the earlier conversation. The record and message indices below identify that complete history. Message indices are zero-based.

Examples 1 and 2 show the same player's round 5 choice before and after learning that a participant ticket won. The initial 2/8 allocation offsets the computer's 8/2 allocation. The later 5/5 reply changes that choice. Examples 3 and 4 contain valid ticket counts but violate the single-object response instruction through code fences and repeated objects, respectively.

```
[EXAMPLE 1]
Model openai/gpt-6-luna
Condition 4A_4AR
Participant number 3
Round 5
JSONL line 61
Prompt message index 14
Reply message index 15

[PROMPT]
Round 5. In this round, the computer has already assigned 8 of its 10 tickets to
  Household A and 2 to Household B. Its allocation cannot be changed.

Assign your 10 tickets between Household A and Household B based on the information
  you have received for this round.

Respond with a single JSON object and nothing else: {"A": <number of your 10 tickets
  to Household A>, "B": <number of your 10 tickets to Household B>}. Use whole numbers
  from 0 to 10 that sum to 10.

[RAW REPLY]
{"A":2,"B":8}

[EXAMPLE 2]
Model openai/gpt-6-luna
Condition 4A_4AR
Participant number 3
Round 5
JSONL line 61
Prompt message index 23
Reply message index 24

[PROMPT]
Earlier in Round 5, you assigned 2 of your tickets to Household A and 8 to Household
  B. The computer assigned 8 tickets to A and 2 to B. You now know that the winning
  ticket is one of your tickets 1-10, but you do not know its specific number.

Review your earlier allocation. You may keep it or change it. Submit the allocation
  of your 10 tickets that you want to use for this round. To keep your earlier choice,
  submit the same numbers again.

Respond with a single JSON object and nothing else: {"A": <number of your 10 tickets
  to Household A>, "B": <number of your 10 tickets to Household B>}. Use whole numbers
  from 0 to 10 that sum to 10.
```

[RAW REPLY]
{"A":5,"B":5}

[EXAMPLE 3]
Model gemini-3.5-flash-lite
Condition 4A_4AR
Participant number 0
Round 1
JSONL line 21
Prompt message index 2
Reply message index 3

[PROMPT]
Round 1. In this round, the computer has already assigned 7 of its 10 tickets to Household A and 3 to Household B. Its allocation cannot be changed.

Assign your 10 tickets between Household A and Household B based on the information you have received for this round.

Respond with a single JSON object and nothing else: {"A": <number of your 10 tickets to Household A>, "B": <number of your 10 tickets to Household B>}. Use whole numbers from 0 to 10 that sum to 10.

[RAW REPLY]
```json
{"A": 5, "B": 5}
```

[EXAMPLE 4]
Model recursive/flint-v1.0:free
Condition 4A_4AR
Participant number 0
Round 3
JSONL line 1
Prompt message index 8
Reply message index 9

[PROMPT]
Round 3. In this round, the computer has already assigned 10 of its 10 tickets to Household A and 0 to Household B. Its allocation cannot be changed.

Assign your 10 tickets between Household A and Household B based on the information you have received for this round.

Respond with a single JSON object and nothing else: {"A": <number of your 10 tickets to Household A>, "B": <number of your 10 tickets to Household B>}. Use whole numbers from 0 to 10 that sum to 10.

[RAW REPLY]
{"A": 5, "B": 5}{"A": 5, "B": 5}